

Relazione Progetto Sistemi Distribuiti - Homework 3

Gruppo di lavoro: Salvatore Rinaudo (1000084774), Paolo Alfò (1000030219)

Sommario

Abstract	2
Descrizione delle principali funzionalità implementate:	3
At Most-Once e Idempotenza	3
Circuit Breaker	3
Comunicazione Inter-Service e Validazione Sincrona tramite gRPC	5
Deployment su Kubernetes:	6
Sistema di Notifica Asincrono con Kafka	6
White-box Monitoring con Prometheus:	8
Integrazione con Alertmanager:	8
Integrazione con l'esportatore del nodo:	9
Principali microservizi:	9
Microservizio: User Manager	11
Microservizio: Data Collector	12
Monitoraggio Infrastrutturale: Node Exporter	13
SLA BREACH DETECTOR	13
Configurazione Iniziale	14
Campionamento e Vincoli	14
Interazioni Tra I Componenti E Flussi Dati	15
Flusso di Registrazione e Idempotenza (At-Most-Once)	15
Configurazione Interessi e Validazione Sincrona	16
Ciclo di Raccolta Dati e Circuit Breaker	16
Analisi Allarmi e Notifica Asincrona	17
Monitoraggio delle Performance (White-box Monitoring)	17
Struttura Dei Database	20
Database: user_db	20
Database: data_db	20
Gestione dei permessi e inizializzazione	22

Abstract

Il progetto consiste nello sviluppo di un sistema distribuito basato su microservizi per gestire utenti e visualizzare voli. L'applicazione consente agli utenti di registrarsi, aggiungere le proprie preferenze degli aeroporti e visualizzare i diversi voli per un dato aeroporto, grazie al collegamento con l'API di OpenSky. La comunicazione tra server e il client si basa su una comunicazione gRPC, che consente di trasmettere richieste e risposte in modo rapido ed efficiente. Il sistema implementa la politica at-most-once, che assicura che alcune richieste vengano processate al massimo una sola volta, evitando esecuzioni multiple della stessa richiesta, come ad esempio le operazioni di registrazione o aggiungere preferenze. Gli utenti potranno configurare soglie personalizzate (high-value e low-value) per i loro aeroporti preferiti che verranno monitorate e notificate attraverso Apache Kafka, che garantisce un'elaborazione scalabile e asincrona dei dati e delle notifiche. Quando le preferenze vengono aggiornate, il DataCollector invierà un messaggio a un topic Kafka chiamato to-alert-system per notificare l'aggiornamento dei dati. L'AlertSystem successivamente analizzerà il database e invierà notifiche tramite Kafka su un altro topic, to-notifier, per gli utenti il cui numero di voli ha oltrepassato le soglie stabilite. Infine, l'AlertNotifierSystem invierà le notifiche agli utenti tramite e-mail, grazie all'utilizzo del server google SMTP. Viene deciso rispetto all'implementazione precedente di distribuire i microservizi utilizzando Kubernetes, creando un cluster locale tramite lo strumento Kind. Kubernetes permetterà di orchestrare i vari componenti del sistema. Ogni microservizio sarà eseguito in un container isolato, garantendo resilienza e la capacità di gestire

eventuali guasti senza compromettere l'intero sistema. Introdotto nella nuova versione il monitoraggio delle performance dei microservizi, tramite il whitebox monitoring utilizzando Prometheus. Ogni microservizio esporrà due tipi di metriche attraverso gli exporter Prometheus: una metrica di tipo GAUGE, e una metrica di tipo COUNTER, che terrà traccia di eventi incrementali come il numero di richieste ricevute o il numero di errori. La combinazione dei microservizi, Kafka, Kubernetes e Prometheus, offre un sistema robusto, scalabile e performante, in grado di gestire notifiche personalizzate e carichi variabili con efficienza e affidabilità.

Descrizione delle principali funzionalità implementate:

At Most-Once e Idempotenza

Per garantire la coerenza dei dati, il microservizio User Manager implementa la politica at-most-once sulle operazioni critiche come la registrazione utente. Questo avviene tramite l'uso di un X-Request-ID univoco inviato dal client; il server memorizza gli ID elaborati nella tabella processed_requests e, in caso di duplicati, restituisce la risposta salvata senza rieseguire l'operazione.

Circuit Breaker

L'implementazione del pattern **Circuit Breaker** tramite la libreria pybreaker nel microservizio Data Collector permette di garantire la resilienza del sistema quando si interagisce con API esterne come quella di OpenSky Network

- **Configurazione Specifica:** Nel file app.py del data_collector, il Circuit Breaker è inizializzato con parametri precisi: fail_max=3 e

reset_timeout=60. Questo significa che, se le chiamate all'API di OpenSky falliscono per 3 volte consecutive, il circuito passa dallo stato "Closed" (chiuso, funzionamento normale) allo stato "Open".

- **Gestione dello Stato "Open":** Una volta che il circuito è aperto, il sistema smette immediatamente di tentare nuove richieste verso l'endpoint esterno per un periodo di 60 secondi. Durante questo intervallo, viene sollevata un'eccezione `pybreaker.CircuitBreakerError`, catturata dal worker che stampa il log "CIRCUIT BREAKER OPEN".
- **Scopo della Resilienza:** Questa implementazione evita il fenomeno del "cascading failure" (fallimento a catena). Impedisce che il thread worker rimanga bloccato in attesa di timeout infiniti da parte di un servizio esterno lento o non disponibile, preservando le risorse locali del microservizio.
- **Integrazione con OpenSky:** Il breaker è applicato specificamente alla funzione `fetch_with_breaker`, che gestisce le richieste HTTP GET verso gli endpoint dei voli di OpenSky. È particolarmente utile per gestire il **Rate Limiting** imposto dai server di OpenSky, permettendo al sistema di fermarsi per un dato tempo prima di riprovare.
- **Monitoraggio:** L'apertura del circuito può essere indirettamente monitorata tramite Prometheus osservando la metrica `last_fetch_duration_seconds` o l'assenza di incrementi nella metrica `flights_fetched_total` per gli aeroporti interessati durante il periodo di blocco.

Comunicazione Inter-Service e Validazione Sincrona tramite gRPC

Per la gestione della coerenza dei dati tra i domini User Manager e Data Collector, è stato adottato un modello di comunicazione sincrona basato sul framework **gRPC**. Questa scelta è motivata dalla necessità di garantire una validazione forte in fase di configurazione degli interessi utente, mantenendo al contempo prestazioni elevate.

L'architettura del servizio è caratterizzata da:

- **Definizione dell'Interfaccia:** Tramite il linguaggio di IDL (Interface Definition Language) **Protocol Buffers**, è stato definito il servizio UserService e il metodo CheckUserExists. Questo assicura una tipizzazione statica dei messaggi e un disaccoppiamento tra l'implementazione del client e del server.
- **Efficienza Protocollare:** A differenza delle tradizionali architetture REST/JSON, l'impiego di gRPC su trasporto HTTP/2 permette una serializzazione binaria dei dati, riducendo sensibilmente l'overhead di rete e la latenza di comunicazione.
- **Service Discovery:** All'interno del cluster Kubernetes, la risoluzione dell'endpoint di destinazione avviene tramite il servizio DNS nativo, puntando al record user-manager-service sulla porta 50051.

Dunque, il flusso di validazione garantisce che l'inserimento di un nuovo interesse nel database data_db avvenga esclusivamente previa conferma dell'esistenza dell'entità utente nel database user_db, preservando l'integrità logica del sistema senza introdurre colli di bottiglia prestazionali.

Deployment su Kubernetes:

Rispetto alla precedente implementazione, l'applicazione è distribuita su piattaforma Kubernetes, utilizzando Kind (Kubernetes in Docker) per il cluster locale. I seguenti oggetti Kubernetes sono stati utilizzati per il deployment:

- Deployment dei microservizi: Ogni microservizio (DataCollector, AlertSystem, AlertNotifierSystem, ecc) è stato containerizzato e gestito tramite Kubernetes. Le immagini Docker vengono caricate nel cluster con i comandi `kind load docker-image`.
- Service dei microservizi: Ad ogni pod configurato tramite il file Deployment è associato un Service che rappresenta un'astrazione dell'indirizzo ip del pod.

Sistema di Notifica Asincrono con Kafka

Una delle funzionalità centrali dell'Homework 3 è l'implementazione di un sistema di notifiche che avvisa l'utente quando il volume di traffico aereo di un aeroporto monitorato supera o scende sotto le soglie definite. Questo sistema è basato su Apache Kafka, una piattaforma di messaggistica asincrona che funge da spina dorsale per la comunicazione tra i microservizi.

L'architettura basata su Kafka offre diversi vantaggi chiave:

- Persistenza e Affidabilità: Kafka garantisce che i messaggi non vadano persi anche in caso di guasto temporaneo di un microservizio, permettendo il recupero dei dati una volta ripristinato il sistema.
- Scalabilità: Sebbene il carico di lavoro attuale sia gestibile da un singolo broker, la struttura è pronta per crescere orizzontalmente

aggiungendo ulteriori partizioni o broker in scenari di traffico elevato.

Per questa implementazione, è stato configurato un singolo broker Kafka (all'interno del cluster Kubernetes o tramite Docker Compose) per bilanciare la semplicità di manutenzione con la solidità necessaria per un ambiente di sviluppo locale.

Flusso di funzionamento del sistema:

1. **Data Collector:** Questo microservizio recupera periodicamente i dati sui voli dalle API di OpenSky. Dopo aver aggiornato il database locale (`data_db.flights`), agisce come producer inviando un messaggio al topic `to-alert-system`. Il messaggio contiene il codice ICAO dell'aeroporto, il numero totale di voli rilevati e il timestamp dell'operazione.
2. **Alert System:** Agisce sia come consumer che come producer. Ascolta costantemente il topic `to-alert-system` e, per ogni aggiornamento ricevuto, interroga la tabella `interests` nel database per verificare se il numero totale di voli viola le soglie (`high_value` o `low_value`) impostate dall'utente. In caso di superamento, genera un payload di allerta e lo pubblica sul topic `to-notifier`.
3. **Alert Notifier:** Questo servizio è un consumer dedicato al topic `to-notifier`. Una volta ricevuto un messaggio di allerta, estrae i dettagli (email del destinatario, oggetto e corpo del messaggio) e procede all'invio fisico della notifica tramite SMTP, utilizzando i server di Google (Gmail) come configurato nel codice. L'email informa l'utente se il traffico aereo dell'aeroporto di interesse è superiore alla soglia massima o inferiore alla minima.

White-box Monitoring con Prometheus:

È stato introdotto nell'implementazione seguente un sistema di monitoraggio delle performance con Prometheus.

I microservizi forniscono almeno due di queste metriche:

- Una metrica di tipo GAUGE per misurare variabili dinamiche come il tempo di risposta o il tempo di aggiornamento.
- Una metrica di tipo COUNTER per monitorare eventi incrementali come il numero di richieste ricevute, il numero di errori o il numero di invocazioni di una funzione.

Le metriche sono etichettate con delle label che contengono il nome del servizio e il nodo che ospita l'esecuzione, consentendo un monitoraggio dettagliato e la possibilità di analizzare le performance per singolo microservizio.

Integrazione con Alertmanager:

Il sistema utilizza Alertmanager per la gestione degli allarmi, garantendo una risposta tempestiva a eventi critici. La configurazione del sistema prevede:

- Rilevamento automatico delle anomalie nelle metriche raccolte da Prometheus, come servizi inattivi o un aumento anomalo degli errori.
- Routing personalizzato degli allarmi che inviano, come configurato all'indirizzo predefinito utilizzando il server google SMTP.

L'integrazione con Alertmanager offre numerosi vantaggi per la gestione degli avvisi e il monitoraggio del sistema.

Integrazione con l'esportatore del nodo:

Il sistema include Node Exporter, uno strumento che raccoglie informazioni sulle risorse di sistema come CPU, memoria, rete e utilizzo dello spazio di archiviazione da ciascun nodo del cluster. Ciò ti consente di monitorare non solo le prestazioni dei tuoi microservizi, ma anche le risorse della tua infrastruttura, offrendoti un quadro completo dello stato di salute del tuo sistema.

Node Exporter garantisce che tutte le risorse siano monitorate in modo uniforme. Le informazioni raccolte vengono inviate a Prometheus ed elaborate per fornire un quadro accurato delle prestazioni delle risorse di sistema.

Principali microservizi:

API Gateway (Nginx): Rappresenta l'unico punto di ingresso per il client. Il suo compito principale è quello di ricevere le richieste HTTP e instradarle verso il microservizio corretto in base al percorso dell'URL: le chiamate a /users vengono inviate allo User Manager, mentre quelle a /interests, /flights e /analysis sono dirette al Data Collector.

User Manager: È il servizio responsabile della gestione del ciclo di vita degli utenti. Gestisce la registrazione e l'eliminazione dei profili nel database user_db e implementa la logica di idempotenza "at-most-once" per evitare registrazioni duplicate. Inoltre, espone un server **gRPC** (porta 50051) che permette agli altri servizi di verificare rapidamente se un utente è registrato.

Data Collector: È il cuore operativo per il recupero dei dati aeronautici. Un thread worker interno interroga periodicamente le API di **OpenSky Network** per ottenere informazioni sui voli degli aeroporti monitorati. Implementa un **Circuit Breaker** per gestire i fallimenti

delle API esterne e, dopo aver salvato i dati nel database `data_db`, invia un aggiornamento al topic Kafka `to-alert-system`.

Alert System: Questo microservizio monitora il flusso di dati proveniente da Kafka. Consuma i messaggi dal topic `to-alert-system` e confronta il numero di voli rilevati con le soglie massime (`high_value`) e minime (`low_value`) impostate dagli utenti. Se viene rilevata una violazione della soglia, genera un evento di allerta sul topic `to-notifier`.

Alert Notifier: Si occupa esclusivamente dell'invio fisico delle notifiche. Consuma i messaggi di allerta dal topic `to-notifier` e utilizza il protocollo **SMTP** per inviare email reali agli utenti tramite i server di Gmail, utilizzando le credenziali configurate in modo sicuro nel sistema.

Database (MySQL): Fornisce la persistenza dei dati per l'intero sistema. È suddiviso in due database logici: `user_db`, che contiene l'anagrafica utente e i log delle richieste per l'idempotenza, e `data_db`, che memorizza le preferenze di monitoraggio (interessi) e lo storico dei voli scaricati.

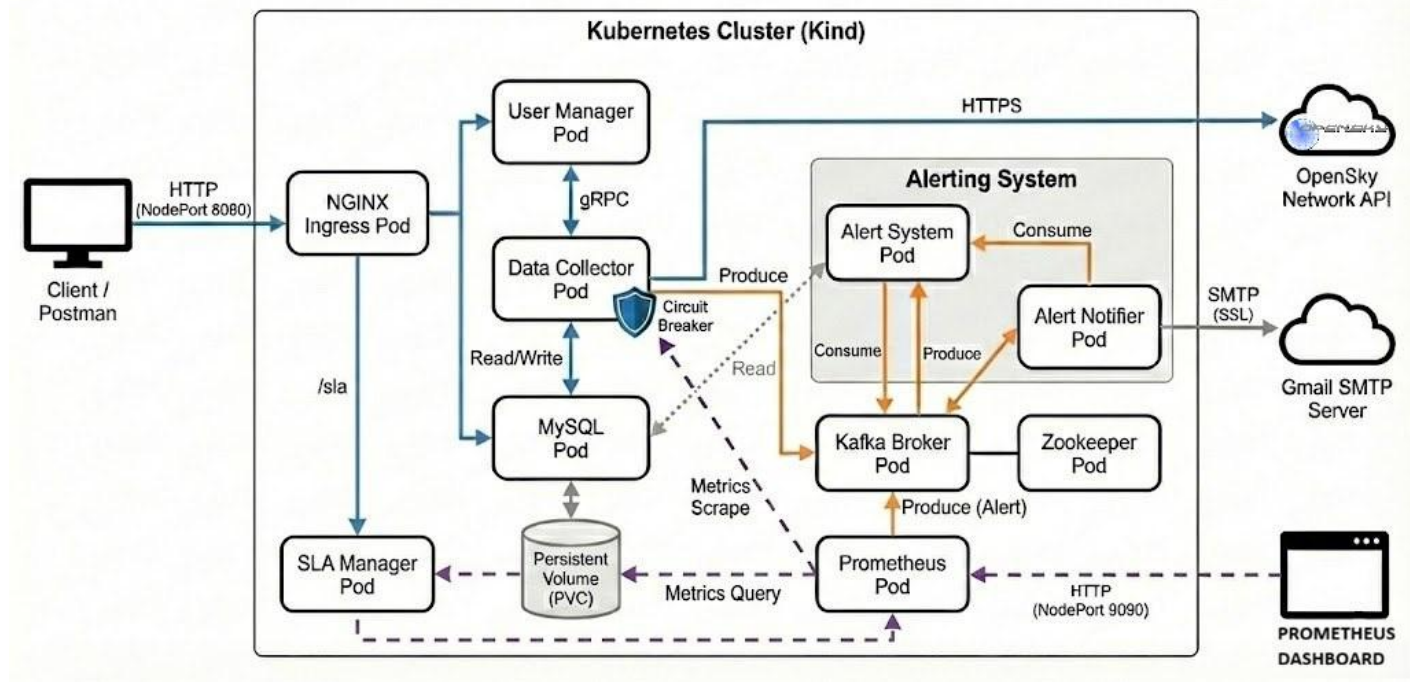
Kafka & Zookeeper: Fungono da infrastruttura per la comunicazione asincrona. Kafka gestisce i topic (`to-alert-system` e `to-notifier`) che permettono il disaccoppiamento tra la raccolta dati, l'analisi delle soglie e l'invio delle notifiche, mentre Zookeeper si occupa del coordinamento del broker Kafka.

Prometheus: È il componente dedicato al monitoraggio "white-box". Effettua lo scraping periodico delle metriche (Counter e Gauge) esposte da ogni microservizio tramite l'endpoint `/metrics`, permettendo di analizzare in tempo reale il numero di richieste, gli utenti registrati e le performance del recupero dati.

In base al codice sorgente del progetto **HW3** (Sky Monitor), ecco la lista dettagliata delle metriche implementate per ogni microservizio,

completa di tipologia ed etichette (labels) utilizzate per il monitoraggio white-box con Prometheus.

Homework 3 Architecture: Kubernetes with Monitoring & Resilience



Microservizio: User Manager

Questo servizio monitora il volume delle richieste e lo stato degli utenti registrati nel database.

- **REQUEST_COUNT**

- **Nome della metrica:** user_requests_total.
- **Tipo:** Counter.
- **Etichette:** service, node, endpoint, method.
- **Descrizione:** Conta il numero totale di richieste HTTP ricevute dal servizio. Permette di analizzare il carico di lavoro suddiviso per endpoint (es. /users) e metodo (POST, DELETE).

- **REGISTERED_USERS**

- **Nome della metrica:** users_registered_count.

- **Tipo:** Gauge.
- **Etichette:** service, node.
- **Descrizione:** Fornisce un'istantanea del numero totale di utenti registrati nel database user_db. Viene aggiornata dinamicamente a ogni operazione di scraping o registrazione.

Microservizio: Data Collector

Il monitoraggio in questo microservizio si concentra sull'attività di recupero dati dalle API esterne e sulle relative performance.

- **FLIGHTS_FETCHED_TOTAL**

- **Nome della metrica:** flights_fetched_total.
- **Tipo:** Counter.
- **Etichette:** service, node, airport.
- **Descrizione:** Registra il numero cumulativo di voli scaricati con successo tramite le chiamate alle API di OpenSky. L'etichetta airport permette di monitorare l'attività per ogni singolo scalo monitorato.

- **LAST_FETCH_DURATION**

- **Nome della metrica:** last_fetch_duration_seconds.
- **Tipo:** Gauge.
- **Etichette:** service, node, airport.
- **Descrizione:** Misura il tempo (in secondi) impiegato per completare l'ultima operazione di fetch dei dati. È utile per rilevare latenze anomale nelle comunicazioni con i server di OpenSky.

Monitoraggio Infrastrutturale: Node Exporter

Viene utilizzato per raccogliere metriche hardware e di sistema operativo da ciascun nodo del cluster Kubernetes.

- **Metriche di Risorsa**
 - **Tipo:** Gauge e Counter (varie).
 - **Descrizione:** Include il monitoraggio dell'utilizzo della CPU (per core), della memoria RAM occupata, del traffico di rete e dello spazio disponibile sui dischi montati (PVC).

SLA BREACH DETECTOR

Lo SLA Breach Detector è un microservizio progettato per il monitoraggio della qualità del servizio. Il suo obiettivo primario è l'identificazione di violazioni persistenti dei Service Level Agreement (SLA) basandosi su metriche di tipo GAUGE esposte tramite Prometheus.

Il servizio funge da componente di monitoraggio attivo e si integra con l'ecosistema esistente come segue:

- **Prometheus:** Il servizio interroga le API HTTP di Prometheus per estrarre i valori correnti delle metriche tramite query PromQL.
- **Kafka:** Utilizzato come canale di notifica per emettere eventi di breach verso il topic to-notifier.
- **Alert Notifier:** Riceve i messaggi da Kafka per l'inoltro finale tramite email.

- **Nginx (API Gateway):** Espone gli endpoint REST del servizio sotto il prefisso /sla per la gestione esterna.

Il microservizio implementa una logica di rilevamento basata su tre fasi critiche:

Configurazione Iniziale

All'avvio, il sistema carica un file di configurazione (sla_config.yaml) che definisce per ogni metrica: nome, query PromQL, e le soglie di tolleranza min e max.

Campionamento e Vincoli

Il monitoraggio avviene tramite un job periodico eseguito ogni Tcheck secondi. Per garantire la coerenza dei dati rispetto allo scraping di Prometheus, è stato applicato il seguente vincolo:

$$T_{check} > 5 \times T_{scrap}$$

Dato che l'intervallo di scraping nel sistema è di 15 secondi, il parametro è stato impostato a 90 secondi.

Per evitare falsi positivi dovuti a picchi momentanei, una violazione viene registrata solo se risulta persistente. Un evento di **SLA Breach** viene generato se almeno **3 campioni consecutivi** risultano al di sotto del valore minimo o al di sopra del valore massimo definito.

Il servizio è distribuito in ambiente **Kubernetes** con le seguenti caratteristiche:

- **Container:** Basato su immagine Python 3.9-slim.

- **Configurazione:** Il file delle soglie è montato come ConfigMap per permettere modifiche senza alterare l'immagine del container.
- **Affidabilità:** Utilizza variabili d'ambiente per localizzare i servizi Prometheus e Kafka all'interno del cluster.

L'implementazione dello SLA Breach Detector aggiunge un livello di osservabilità critica al progetto, permettendo non solo di reagire a guasti immediati, ma di monitorare il degradamento prestazionale nel tempo, garantendo il rispetto dei parametri di qualità del software stabiliti.

Interazioni Tra I Componenti E Flussi Dati

Le interazioni all'interno del sistema sono state progettate per garantire scalabilità, bassa latenza e affidabilità, utilizzando una combinazione di protocolli REST, gRPC e messaggistica asincrona tramite Kafka.

Flusso di Registrazione e Idempotenza (At-Most-Once)

Questo flusso garantisce che i dati dell'utente siano persistiti correttamente senza duplicazioni in caso di errori di rete.

- **Utente (Client) → POST /users (con X-Request-ID) → API Gateway (Nginx):** Il client avvia la registrazione inviando un identificativo univoco nell'header per l'idempotenza.
- **API Gateway (NGINX) → User Manager:** NGINX instrada la richiesta al servizio preposto sulla porta 5000.
- **User Manager → Verifica Request-ID → User DB:** Il server controlla nella tabella `processed_requests` se l'operazione è già stata eseguita.

- **User Manager** →INSERT user →**User DB**: Se l'ID è nuovo, i dati vengono salvati nel database user_db.
- **User Manager** →Risposta 201 Created →**Utente**: Conferma il successo dell'operazione.

Configurazione Interessi e Validazione Sincrona

In questa fase, il sistema associa un aeroporto a un utente, verificando l'identità tramite gRPC.

- **Utente (Client)** → POST /interests →**API Gateway (Nginx)** →**Data Collector**: L'utente definisce l'aeroporto ICAO e le soglie di allerta.
- **Data Collector** ⇌ gRPC: CheckUserExists ⇌**User Manager**: Il Data Collector interroga lo User Manager (porta 50051) per validare l'esistenza dell'utente prima di procedere.
- **Data Collector** →INSERT interest →**Data DB**: Una volta validato, l'interesse viene salvato nel database data_db.

Ciclo di Raccolta Dati e Circuit Breaker

Il sistema recupera i dati dall'esterno in modo resiliente e li distribuisce internamente.

- **Data Collector (Worker)** →Richiesta Dati →**OpenSky API**: Un thread worker interroga periodicamente l'API esterna.
- **Circuit Breaker** →Monitoraggio Fallimenti →**OpenSky API**: Se l'API esterna restituisce errori ripetuti, il Circuit Breaker apre il circuito per 60 secondi, bloccando le richieste e proteggendo il sistema.
- **OpenSky API** →Dati Voli (JSON) →**Data Collector**: I dati sui voli vengono ricevuti e processati.

- **Data Collector** → Salvataggio → **Data DB**: I singoli voli sono persistiti nella tabella flights.
- **Data Collector** → Produce: total_flights → **Kafka (topic: to-alert-system)**: Il conteggio aggiornato viene inviato al broker Kafka.

Analisi Allarmi e Notifica Asincrona

La logica di alerting è completamente separata dalla raccolta dati per massimizzare l'efficienza.

- **Kafka (topic: to-alert-system)** → Consume → **Alert System**: Il servizio legge i nuovi messaggi dal topic.
- **Alert System** ⇔ SELECT thresholds ⇔ **Data DB**: Il sistema recupera le soglie impostate dall'utente per quell'aeroporto.
- **Alert System** → Verifica Soglia (High/Low) → **Alert System**: Viene eseguito il confronto logico tra i voli attuali e i limiti dell'utente.
- **Alert System** → Produce: alert_payload → **Kafka (topic: to-notifier)**: Se la soglia è violata, viene generato un allarme.
- **Kafka (topic: to-notifier)** → Consume → **Alert Notifier**: Il servizio di notifica preleva l'allarme dalla coda.
- **Alert Notifier** → SMTP Send → **Gmail / Smtip Google**: Viene inviata l'email fisica all'utente finale.

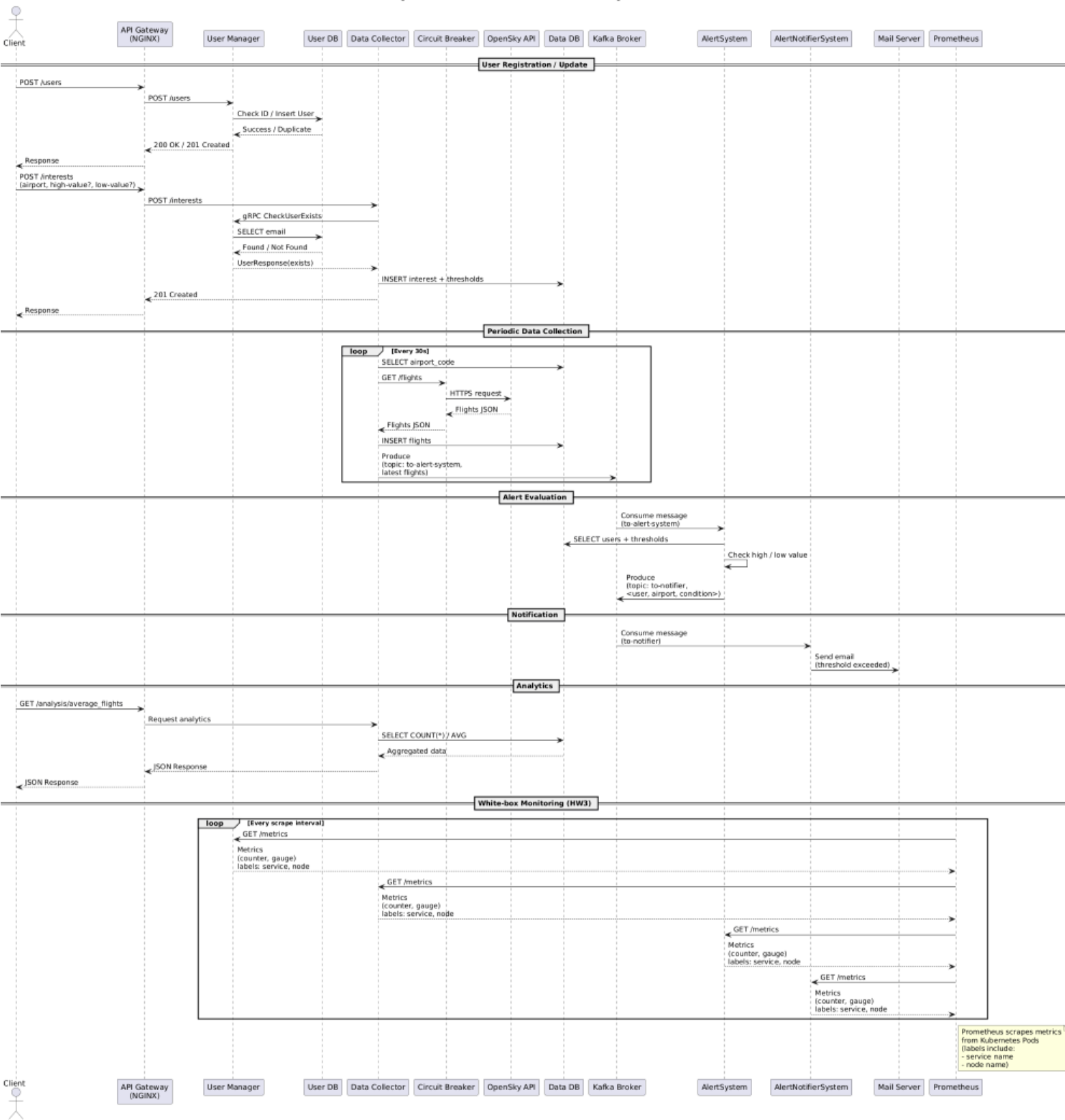
Monitoraggio delle Performance (White-box Monitoring)

Il monitoraggio avviene in parallelo senza interferire con i flussi applicativi.

- **Prometheus** → Scrape: GET /metrics → **Microservizi**: Prometheus interroga periodicamente ogni pod del cluster.

- **Microservizi** → Metriche (Counter/Gauge) → **Prometheus**: Ogni servizio restituisce dati sulle richieste ricevute, errori e performance.
- **Node Exporter** → Risorse Nodo (CPU/RAM) → **Prometheus**: Fornisce metriche infrastrutturali per correlare lo stato dei servizi con le risorse fisiche.

Diagramma delle Interazioni - Homework 3 (Monitoring + Kubernetes)



Struttura Dei Database

La struttura dei database del progetto è suddivisa in due database logici distinti per separare le responsabilità dei microservizi: `user_db` (gestito dallo User Manager) e `data_db` (gestito dal Data Collector).

Database: `user_db`

Questo database è dedicato alla persistenza dei profili utente e alla gestione della politica "at-most-once" per garantire l'idempotenza delle operazioni.

- **Tabella `users`:**

- **email** (VARCHAR, Primary Key): Indirizzo email univoco che identifica l'utente.
- **username** (VARCHAR): Nome visualizzato dell'utente.
- **created_at** (TIMESTAMP): Data e ora di registrazione.

- **Tabella `processed_requests`:**

- **request_id** (VARCHAR, Primary Key): ID univoco della richiesta (UUID) inviato dal client per evitare doppie esecuzioni.
- **operation** (VARCHAR): Tipo di operazione eseguita (es. 'REGISTER_USER').
- **created_at** (TIMESTAMP): Timestamp di quando la richiesta è stata processata.

Database: `data_db`

Questo database gestisce le preferenze di monitoraggio degli utenti e lo storico dei dati aeronautici raccolti.

- **Tabella interests:**

- **id** (INT, Auto Increment, Primary Key): Identificativo univoco della riga.
- **user_email** (VARCHAR): Email dell'utente che ha espresso l'interesse (collegata logicamente a user_db.users).
- **airport_code** (VARCHAR): Codice ICAO dell'aeroporto da monitorare (es. 'LICC').
- **high_value** (INT): Soglia massima di voli oltre la quale scatta l'allerta.
- **low_value** (INT): Soglia minima di voli sotto la quale scatta l'allerta.
- **Vincolo:** Esiste una chiave UNIQUE(user_email, airport_code) per evitare duplicati della stessa associazione.

- **Tabella flights:**

- **id** (INT, Auto Increment, Primary Key): Identificativo univoco del volo registrato.
- **airport_code** (VARCHAR): Aeroporto di riferimento per il dato.
- **icao24** (VARCHAR): Identificativo univoco del transponder del velivolo.
- **callsign** (VARCHAR): Codice identificativo del volo (es. 'AZA123').
- **arrival_airport / departure_airport** (VARCHAR): Aeroporti di origine e destinazione.
- **time** (INT): Timestamp del rilevamento.

- **type** (VARCHAR): Indica se si tratta di un arrivo ('ARRIVAL') o una partenza ('DEPARTURE').
- **Vincolo:** Una chiave UNIQUE su (icao24, time, departure_airport, arrival_airport) evita il salvataggio di duplicati dello stesso evento di volo.

Gestione dei permessi e inizializzazione

I database vengono creati all'avvio tramite lo script init.sql (o mysql-init.yaml in Kubernetes), che configura l'utente app_user con privilegi completi su entrambi gli schemi per consentire ai microservizi di eseguire operazioni CRUD e inizializzare autonomamente le tabelle mancanti.